

CivicPulse

CivicPulse Project Overview

CivicPulse is a Flask-based civic reporting application built to let residents report local infrastructure problems such as potholes, streetlights, drainage issues, garbage, water leaks, and public facility defects. The project is backend-first: the repository contains the API, database model, PostGIS spatial logic, and duplicate-detection logic. The README also expects a separate frontend to consume the API contract in `openapi.yaml`, which is the contract for mobile or web clients.

This repository is organized around a Python backend in the `backend/app` directory, with Postgres/PostGIS for storage and geospatial filtering, Redis for supporting infrastructure, and Docker Compose for local setup. The project is designed for an operational workflow in which a resident reports an issue, a verifier checks it, an authority user takes action, and the system tracks status, duplicate reports, and notifications through the full lifecycle.

1. Registration and login from the resident viewpoint

The user journey starts at signup. A new person submits their name, email, and password, and the system validates the input before creating an account. The registration endpoint rejects missing names, malformed email addresses, weak passwords shorter than eight characters, and invalid area IDs. If the email is already in use, the API returns an `email_taken` error. Importantly, even if a client sends a role in the registration payload, the server ignores it and always creates a resident account. That makes the role assignment explicit and safe, with higher roles created through admin or CLI tools rather than by self-registration.

After signup, the API returns a JWT access token and a user object. The token is later added to requests as an Authorization header. The login endpoint validates the email and password using Werkzeug password hashing, matches the user record in PostgreSQL, and returns a fresh token. The `/auth/me` endpoint lets an authenticated user fetch their own profile and confirms the client is logged in. This is how the app enforces access control: the token identifies the current user, and each protected route checks the user role and session context before allowing the action. The user model stores `id`, `name`, `email`, `password_hash`, `role`, `area_id`, and `creation_timestamp`. The role is constrained to `resident`, `verifier`, `authority`, or `admin`. The database is designed so role changes can be applied immediately on each request, rather than forcing a system restart or hardcoded session state.

2. Core roles and permissions

The system has four user roles. Residents submit and follow reports. Verifiers review reports, approve or reject them, assign cases, and manage duplicate merges. Authorities are the service providers who take ownership of assigned issues and move them from assigned to in progress to resolved. Admins have the broadest permissions and can handle verification, assignments, full user listing, and system administration. The README explicitly states that role checks are

CivicPulse

enforced per request and are not just during login. This is implemented by decorators such as `login_required` and `roles_required`, which verify the current user and the requested role before access is granted.

The core role logic is stored in `app/constants.py`. Roles are defined in a single tuple, and the allowed status transitions are defined as a transition map. For example, reported reports can move to verified or rejected, verified reports can move to assigned, assigned reports to `in_progress`, and `in_progress` reports to resolved. The permission table also checks who is allowed to make each move, using role names rather than raw database enums, making future changes easier.

3. Reference data: areas and categories

The app uses reference tables for civic reporting. Categories are seeded as six built-in problem types: pothole, streetlight, drainage, garbage, water leak, and public facility. They are served through `GET /categories`. Areas are stored in a separate table and may include optional polygon boundaries; the system uses them to map reports to a geographic area or ward/zone. The areas endpoint exposes the list of named areas. This is important because reports are filtered and organized by area, and a symptom can be assigned to a zone automatically when a boundary is present.

A report is also tied to a user-selected category and a spatial point. The database uses PostGIS to store the report location as a Geography POINT with SRID 4326, so the app can measure distances in metres and query within a bounding box or near a point. The project documentation calls out the coordinate order detail: PostGIS stores points as longitude, latitude, while the API returns them as `{lat, lon}`.

4. Reporting a problem: submission and validation

Once the user is signed in, the main action is submitting a civic issue. The report endpoint accepts multipart form data rather than raw JSON because it includes a photo file plus standard fields. The client sends `category_id`, `lat`, `lon`, `description`, and `photo`. The server validates every field strongly: `category_id` must be a valid category, `lat` must be between -90 and 90, `lon` between -180 and 180, `description` under 500 characters, and the uploaded photo must be a JPEG or PNG under 5 MB.

The photo is saved to a local uploads folder by the storage layer, which ensures file type is valid and rejects dangerous paths. The path is stored in the reports table, and the API returns a `photo_url` for the frontend to display. The report also gets a point object created from the coordinates, and the system automatically checks whether the point falls inside any area boundary. If so, the report is assigned to that area automatically. This is a strong feature because it reduces data-entry effort and helps local agencies understand where issues are happening.

When a report is created, the status is set to reported and a status event row is created immediately. This creates the first audit trail for the issue, capturing who changed the status

CivicPulse

and when. The report record includes category, description, created_by, assigned_to, due_date, duplicate status, report_count, AI-suggested category metadata, and timestamps. It is intended to support both operational workflow and reporting analytics.

5. Listing, filtering, sorting, and following reports

The app exposes GET /reports for authenticated users. It supports a wide set of filters and pagination options. These include a bounding box (bbox), category_id, status, area_id, mine=true, following=true, include_duplicates=true, page, and per_page. The backend uses a SQLAlchemy query builder with PostGIS geometry intersection checks to filter reports by geographic area. This means the app can answer map-centered queries such as ?show all reported issues in this ward? or ?show all reports in this bounding box.?

The response format is structured as {items, page, per_page, total}. For each report, the API returns the category details, description, status, geographic coordinates, area_id, photo_url, creator, assignee, due date, report_count, duplicate_of, AI suggestions, follow state, and created/updated times. The follow feature is implemented as a separate table that stores user_id and report_id pairs. A user can follow a report or unfollow it with dedicated endpoints, and the backend prevents duplicate follow rows.

The design allows both personal and collaborative monitoring. A resident can keep an eye on the cases they care about, and the system can notify them whenever an issue changes. This makes the platform more than a simple ticketing app; it supports engagement and accountability around public-service problems.

6. Workflow: verification, assignment, and status progression

The operational heart of CivicPulse is the status workflow. A report starts as reported, then can be moved to verified or rejected by a verifier or admin. The status rules are centralized in app/status_flow.py so the status logic is consistent and not scattered across routes. For example, a report cannot move to an invalid status, cannot jump around the workflow, and cannot be reassigned improperly once work has started.

The allowed transitions are reported -> verified or rejected; verified -> assigned; assigned -> in_progress; in_progress -> resolved. Final states are rejected and resolved. Rejections require a note explaining why the report is invalid. Once a report is assigned, it cannot be reassigned as long as the work has started in_progress. This is enforced by the reassign logic and the business rule that in-progress reports are protected from reassignment.

Assignment is done through POST /reports/{id}/assign. The endpoint accepts assignee_id and an optional due_date. The assignee must be an authority user, the due date must be today or in the future, and the endpoint is used instead of direct status change when moving a report into assigned. The assigned or in-progress report keeps the history of changes through status_events, which records each transition with the actor, timestamp, and note.

The workflow routes also provide GET /reports/{id}/history to fetch status history. This creates a visible audit trail for case managers, allows after-the-fact reasoning, and makes it possible to compute service-time metrics. The project explicitly notes that status_events is intended to support ?days to resolve? reporting and operational analytics in the future.

7. Notifications and case communication

Notifications are a first-class feature, not an afterthought. The notifications model tracks a `user_id`, `report_id`, `message`, `read flag`, and `creation timestamp`. It is created in the same transaction as the status change or assignment that triggers it, so the application never creates a notification for an action that later fails. The system intentionally does not notify the actor about their own action, reducing noise and preserving a cleaner activity stream. Notifications are generated for several classes of events. When a report is verified, rejected, assigned, moved to `in_progress`, or resolved, the author and followers of that case are notified. When a report is merged into another, the author of the merged report gets a message telling them the case has been combined. When a report is reassigned, both the new assignee and the previous assignee can receive specific messages. This keeps the user informed without needing to poll or check the case manually.

The unread notifications count is exposed through `GET /notifications/unread-count`, and users can read one item or all items through dedicated endpoints. This is a polished operational feature and shows that CivicPulse is meant to support a workflow with multiple stakeholders rather than simply storing reports.

8. Duplicate detection, merging, and report count logic

The repository contains a dedicated duplicate detection module in `ml/duplicates/detector.py`. The current implementation is rule-based, not a fully trained AI model: it compares reports by category and by distance, and marks them as duplicates when the same category appears within 200 metres. The backend stores duplicate suggestions separately as a `duplicate_suggestions` table, with a score between 0 and 1 and JSON reasons such as location, category, and time. This is how the app can present `?possible_duplicates?` to a verifier before merging takes place.

Duplicate handling follows a parent-child pattern. A duplicate report stores `duplicate_of` as the parent report id. The parent's `report_count` is recomputed so that it reflects the parent plus all merged child cases. This avoids drift and ensures the count is always accurate. When combining reports, the system moves any already-merged children up to the parent so the merge tree remains shallow and consistent. The verifier can see duplicate suggestions with `GET /reports/{id}/duplicates`, then approve the merge with `POST /reports/{id}/merge` or reverse it with `POST /reports/{id}/unmerge`.

The merge operation enforces strict rules: a report cannot merge into itself, a parent cannot already be a duplicate, only reported or verified reports can be merged, and the parent must still be in an open status. Once a report is merged, it cannot be edited in the usual way; instead, the main parent report becomes the one to update. This is a strong data-integrity safeguard and prevents duplicate issues from being treated as independent cases after the fact.

9. Architecture, infrastructure, and project setup

CivicPulse

The application is built with Flask, SQLAlchemy, Flask-JWT-Extended, Flask-CORS, and Postgres/PostGIS. The app factory creates the configured Flask app in `app/__init__.py`, loads settings from `app/config.py`, initializes the database, JWT, CORS, and migration tools, and registers the API blueprints. The API is split into clear modules: auth, reports, workflow, users, duplicates, notifications, media, health, and reference. This modular structure makes the backend easy to extend without scattering route logic.

The schema contains nine primary tables: users, areas, categories, reports, status_events, follows, notifications, duplicate_suggestions, and hotspots. The database schema and the application model are aligned, and the README explicitly says the model is the source of truth. The schema uses PostGIS Geography columns for points and polygons, plus GiST indexes, which makes geospatial queries efficient and suitable for map-based civic reporting.

The repository includes command-line setup helpers: `flask init-db` initializes PostGIS and creates tables, `flask seed` loads default categories and a default area, and `flask create-user` creates a user with a chosen role. The README also gives a recommended local runtime setup using Docker Desktop, a Python virtual environment, `.env` configuration, strong secret keys, and `pytest`. The project expects Python 3.12 or 3.13 and notes that SQLAlchemy 2.0.54 must be kept pinned because newer versions can break GeoAlchemy2.

10. What is complete and what is still planned

The README clearly separates completed work from future work. The backend supports project setup, database schema, authentication and roles, report submission with photos, report listing with filters, following and status workflows, assignment, authority list, duplicate suggestions and merging, and notifications. Planned next steps include a job queue and worker for background tasks, followed by AI integration for smarter duplicate detection and analytics endpoints such as summary and hotspot calculations. There are also explicit notes that analytics summary and hotspot APIs are not yet built, and the user-role patch endpoint is still planned.

In summary, CivicPulse is a civic issue-oversight system designed for operational reporting and resolution. A resident signs up, logs in, and reports a local problem with a photo and coordinates. The system validates the issue, identifies geographic area, creates the case, then passes it through a strict lifecycle involving verification, assignment, work in progress, and final resolution or rejection. Duplicate reports are detected and merged under admin oversight, while all participants receive focused notifications. The project is a strong example of a modern civic service system that pairs geographic intelligence, workflow management, and user notifications into a single backend-driven pipeline.